PLATAFORMA EDUCATIVA DE CIBERSEGURIDAD

ESPECIFICACIÓN DE ARQUITECTURA, PERSISTENCIA Y MODELADO UML 2.5
SUITE DE DIAGRAMAS (FIGURAS 1 A 7) Y DICCIONARIO DE DATOS

«VERTEX: Plataforma Educativa de Ciberseguridad basada en Teoría de Grafos»

Proyecto de Software:
• Sistema VERTEX (Godot Engine / Data-Driven)
• Modelo de Grafo Ponderado $G = (V, E)$
• Aislamiento Multi-Slot y Respaldo Atómico

Estándares de Ingeniería:
• Especificación UML 2.5 (Casos de Uso, Secuencia, Actividades, Clases, FSM)
• Diagrama Entidad-Relación Físico (Notación Crow's Foot)
• Diccionario de Datos Formal (ANSI/IEEE Std 1059)

Figura 1. Diagrama de Casos de Uso General del Sistema

La **Figura 1** presenta el modelo formal de Casos de Uso General del sistema **VERTEX** bajo el estándar UML 2.5, estructurado estrictamente mediante **verbos de acción en infinitivo** de acuerdo con las buenas prácticas metodológicas. Delimita el alcance perimetral del software frente al actor primario (**Operador**) y los servicios secundarios (**Adversario IA** y el subsistema de **Persistencia I/O**). Los casos de uso unifican la administración de ranuras, la formación en academia y la selección y ejecución de los modos tácticos (Heist, Hacker y Ciberdefensa).

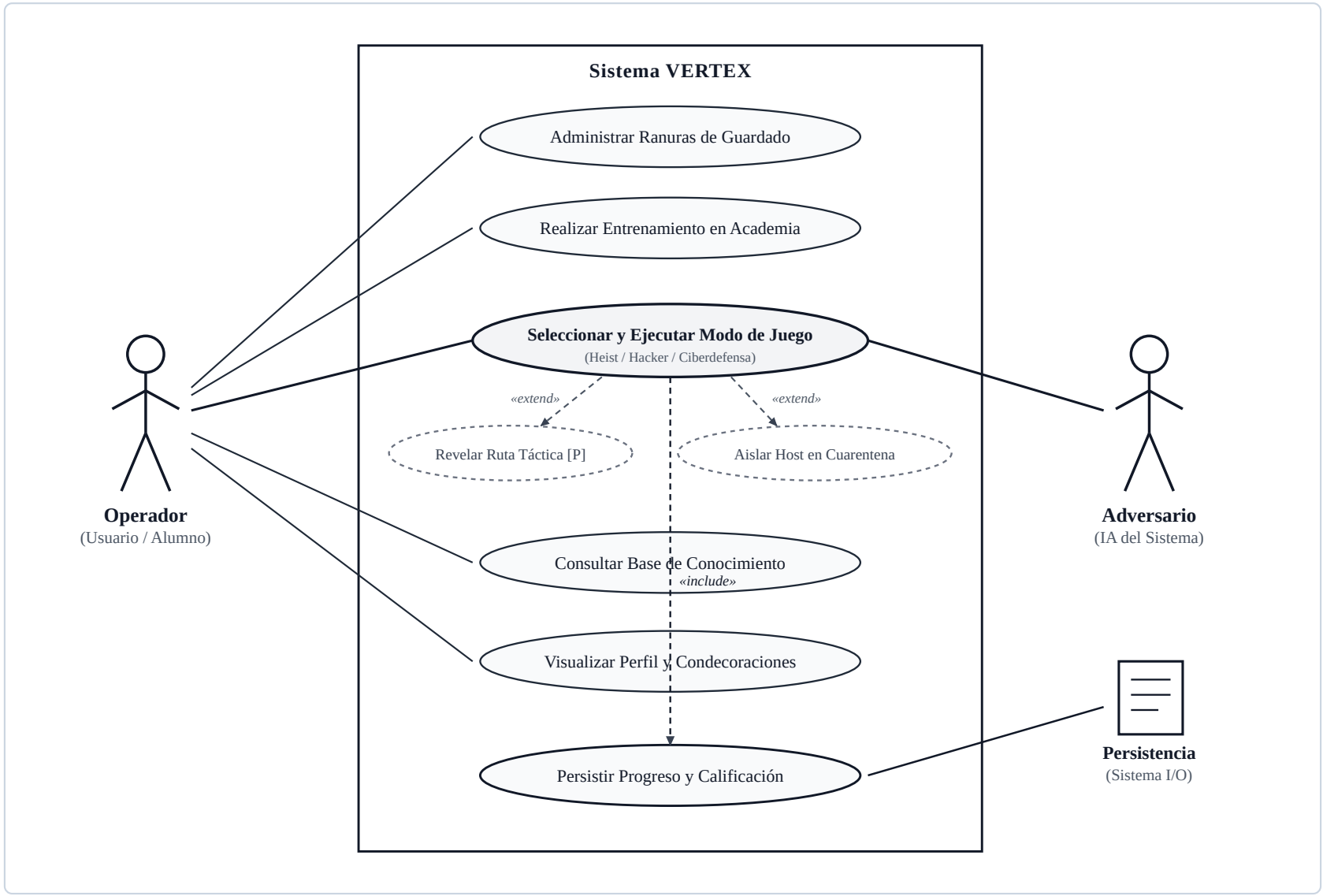


Figura 1. *Diagrama de Casos de Uso General del Sistema VERTEX según especificación UML 2.5.*

Figura 2. Diagrama de Secuencia del Ciclo Reactivo de Ciberdefensa

La **Figura 2** ilustra el **diagrama de secuencia** del ciclo reactivo Blue Team. El flujo temporal modela la recepción del evento de bloqueo en `InputHandler`, la delegación hacia el mediador de lógica (`GameLogic`), el sellado de aristas con costo infinito en `NetworkRuntime`, la resolución automática del turno en `DefenderBrain`, el avance del rastreo forense EDR, la respuesta estocástica del atacante (`move_enemy()`) mediante movimiento lateral o exploit `Bypass`, y la actualización de estados en `GameRenderer`.

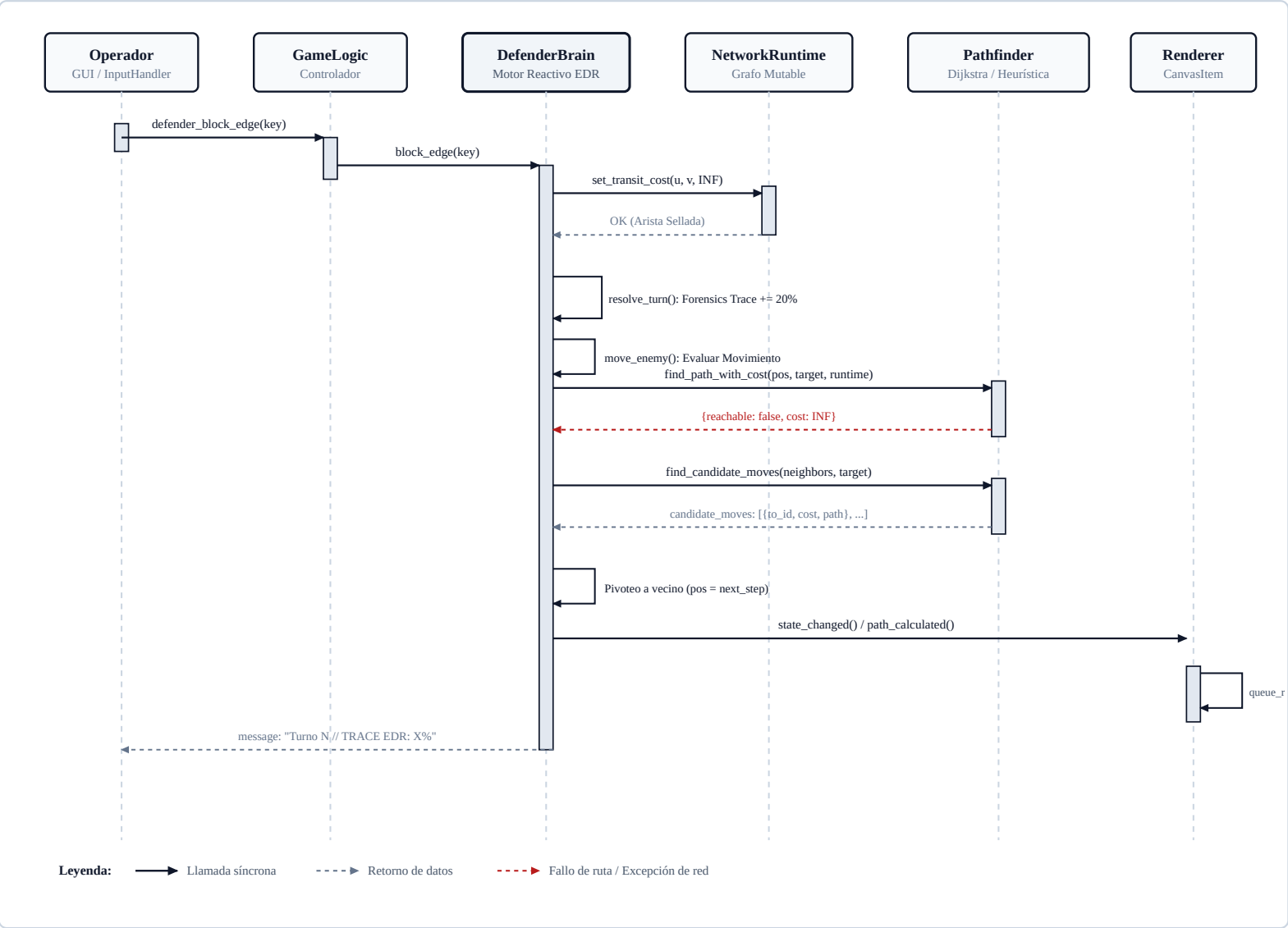


Figura 2. *Diagrama de secuencia del ciclo de respuesta reactiva 1:1 en modo Ciberdefensa.*

Figura 3. Diagrama de Actividades de Ciberdefensa (Procesos)

La **Figura 3** presenta el **diagrama de actividades (flujo de proceso)** del ciclo de defensa perimetral según el estándar UML. Modela el nodo inicial de inicio de asedio, la bifurcación condicional de acciones del operador (bloqueo de arista, colocación de firewall o aislamiento de host en cuarentena), la resolución del turno de la IA mediante búsqueda de camino mínimo, la evaluación de contingencia por exploit `Bypass`, la ramificación por estado de rastreo forense (Trace EDR $\geq 100\%$) y los nodos de victoria o derrota.

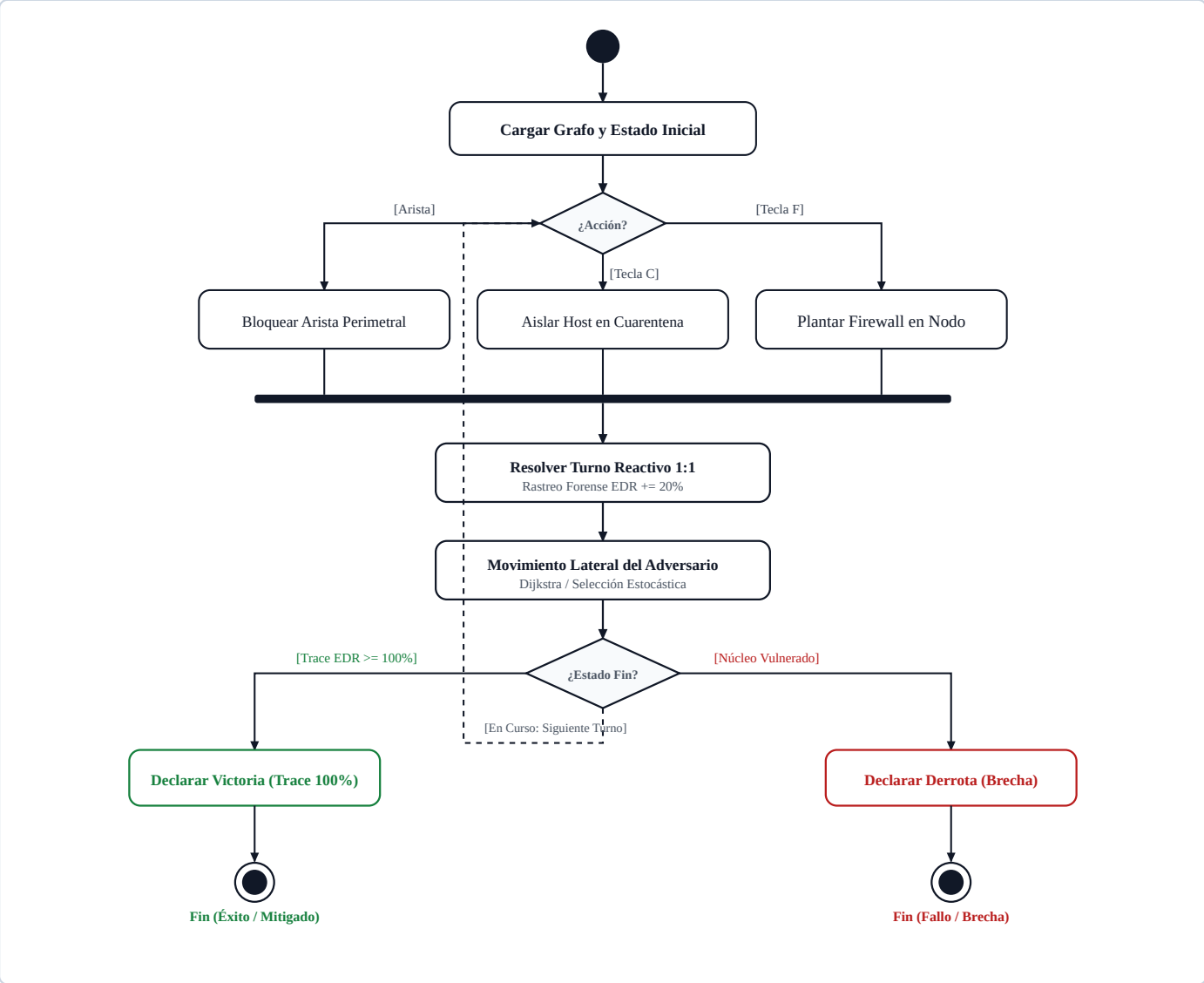


Figura 3. *Diagrama de actividades del ciclo de resolución de Ciberdefensa según estándar UML.*

Figura 4. Diagrama de Actividades de Infiltración Ofensiva (Modo Hacker)

La **Figura 4** ilustra el **diagrama de actividades** del proceso de infiltración ofensiva del modo Hacker (Red Team). Describe la secuencia de decisiones que gobierna el reconocimiento pasivo con `Nmap`, el consumo de exploits (`Bypass`, `Escalate`, `Persist`, `Decoy`), el cálculo y acumulación del nivel de ruido en la red perimetral, la evaluación de umbrales de detección ante la IA centinela (despacho de patrullas y perseguidores), y la finalización del escenario con persistencia de estrellas de maestría.

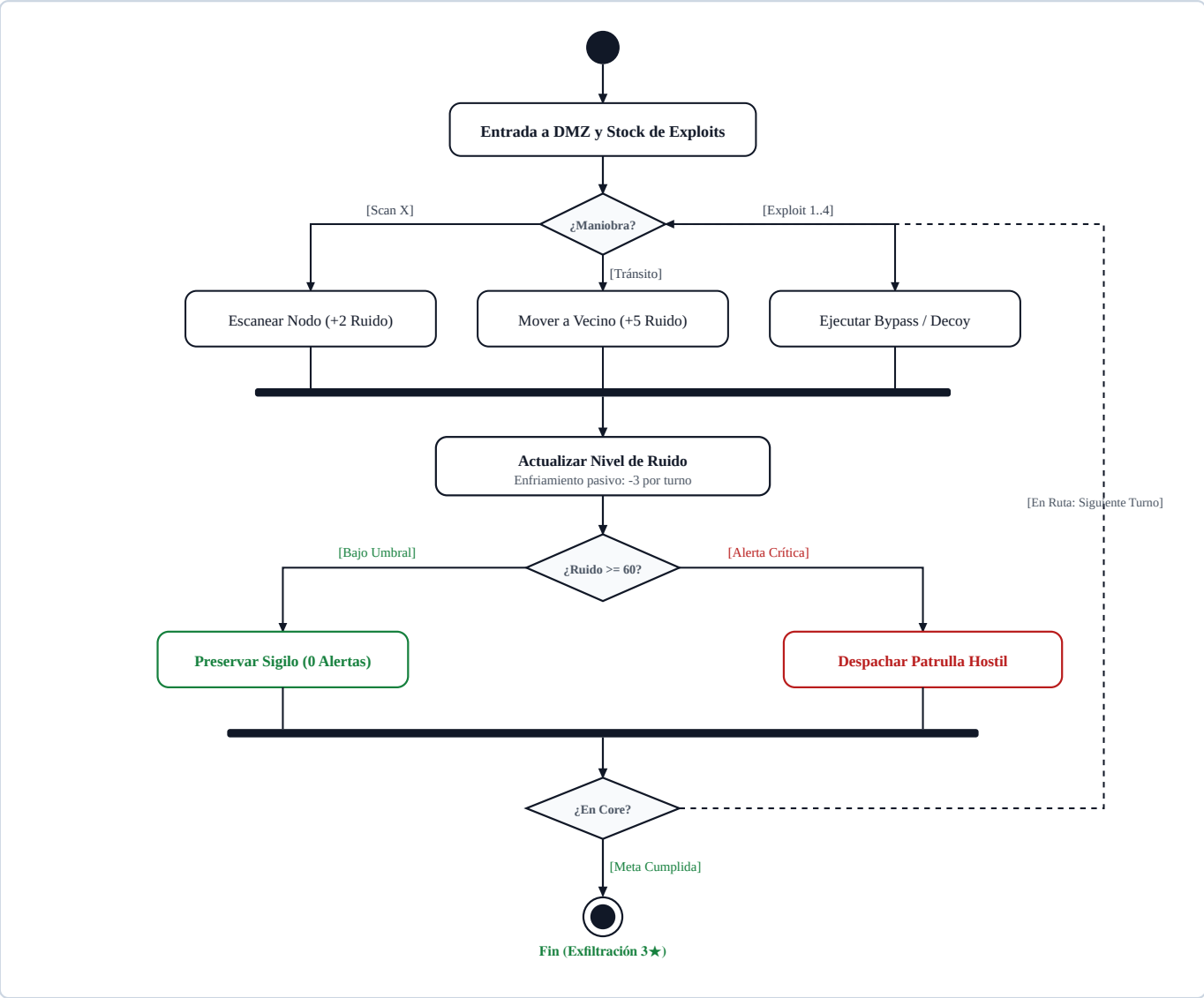


Figura 4. *Diagrama de actividades del ciclo de infiltración y balance de ruido en modo Hacker.*

Figura 5. Diagrama de Clases de Arquitectura (UML 2.5)

La **Figura 5** presenta el **diagrama de clases** del núcleo arquitectónico de **VERTEX** bajo el estándar UML 2.5. Modela la separación estricta de responsabilidades entre los recursos inmutables del grafo (`NetworkGraphResource`, `NetworkNodeResource`, `NetworkEdgeResource`), la capa de mutación en memoria (`NetworkRuntime`), los algoritmos deterministas independientes (`DefensivePathfinder` con Dijkstra y `StrategicAnalyzer` con Edmonds-Karp), y los controladores de dominio y presentación (`DefenderBrain`, `GameLogic` y `GameRenderer`).

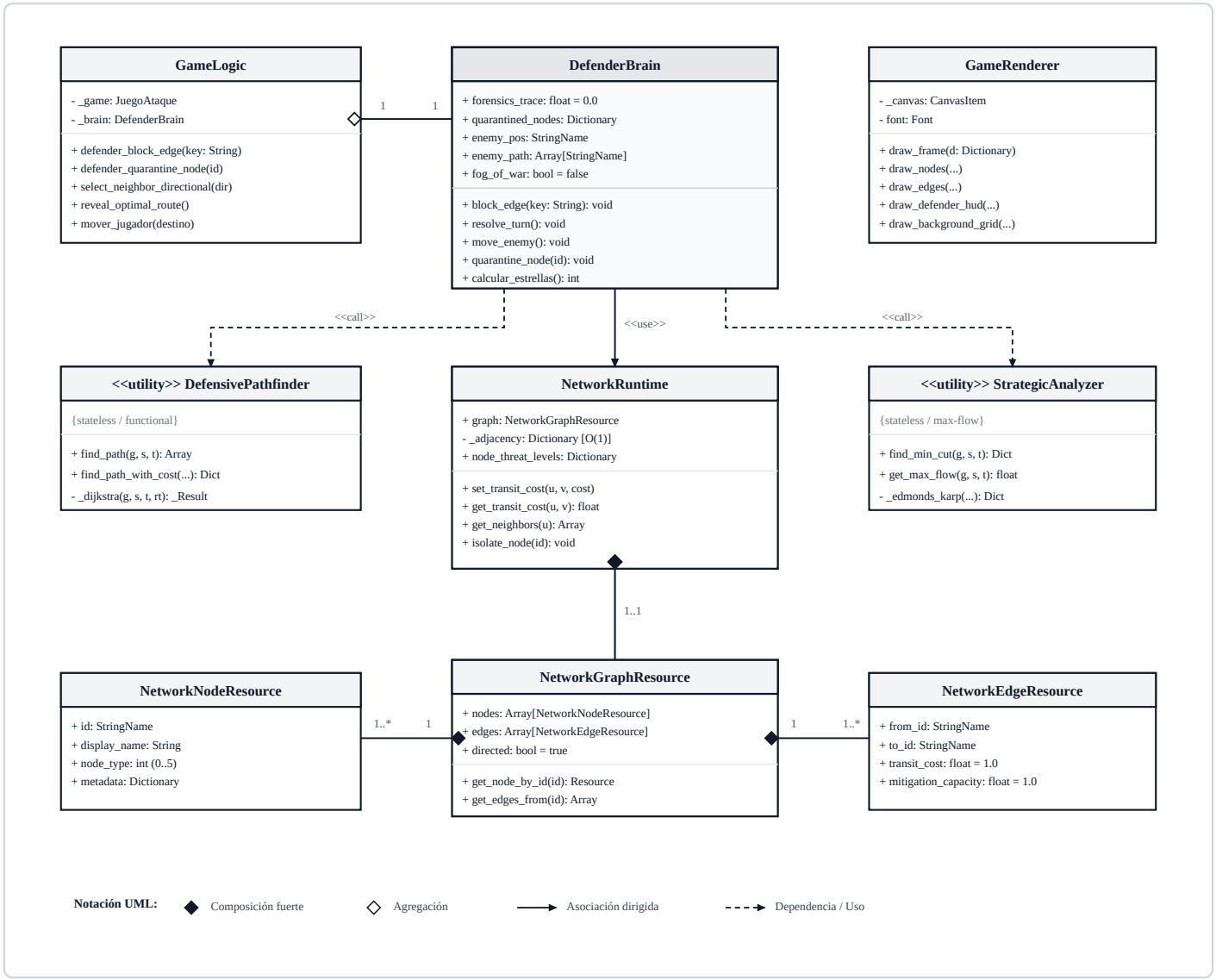


Figura 5. *Diagrama de clases de arquitectura y separación estructural de responsabilidades (UML 2.5).*

Figura 6. Diagrama Entidad-Relación y Modelo de Persistencia (Crow’s Foot)

La **Figura 6** presenta el **diagrama de modelo de datos entidad-relación (ERD / Esquema Físico de Persistencia)** de **VERTEX** bajo notación **Crow’s Foot** (Pata de Gallo). Modela la estructura transaccional de archivos de configuración alojados en `user://`. El meta-registro `SlotMeta` gobierna el puntero de sesión activa sobre las tres ranuras independientes (`SaveSlot`). Cada ranura aísla la entidad de identidad del agente (`Profile`), el progreso (`LevelProgress`), las métricas históricas (`GlobalStats` y `LevelStats`), las condecoraciones (`BadgeRecord`) y los respaldos rotativos (`SlotSnapshot`).

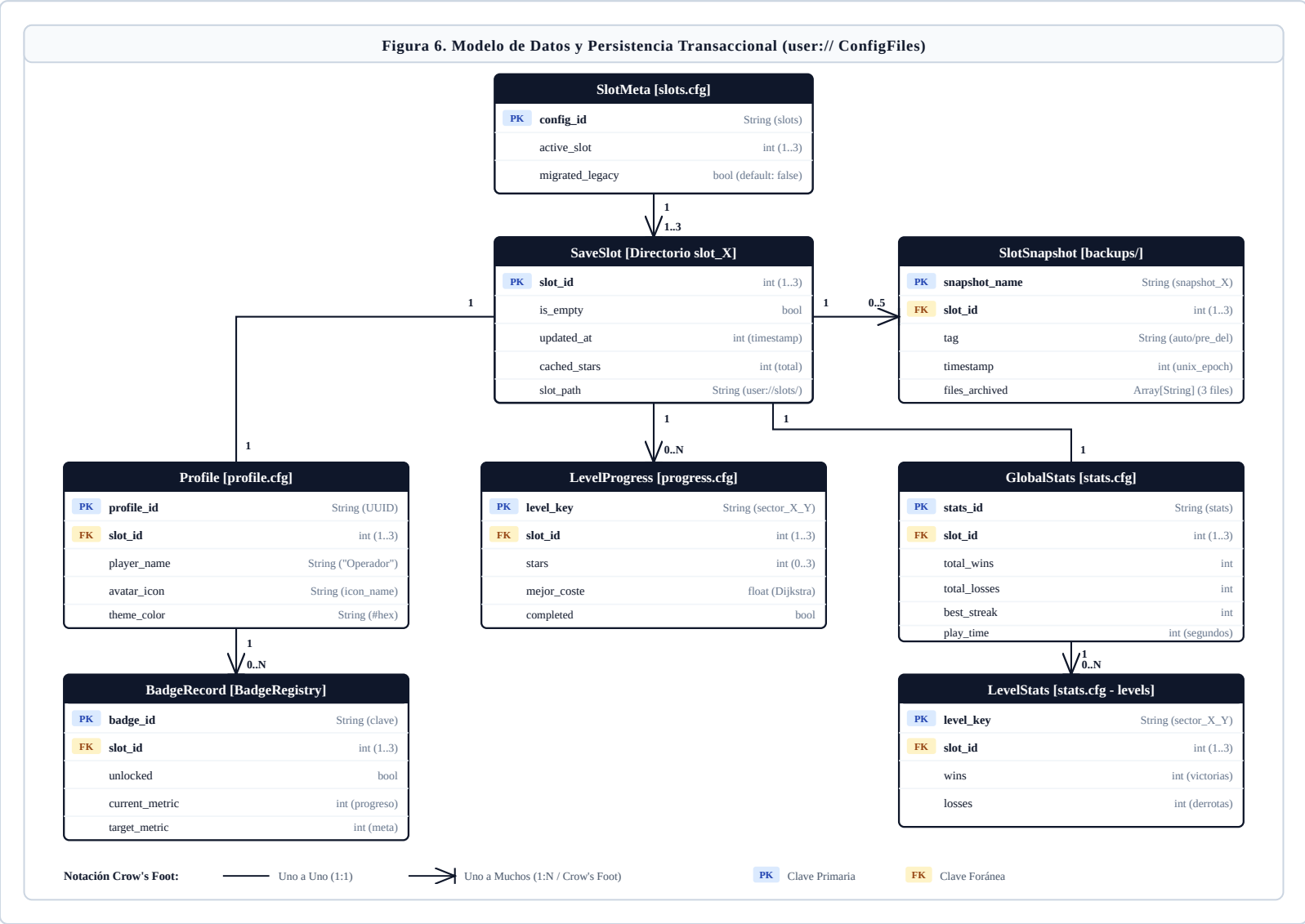


Figura 6. *Diagrama Entidad-Relación y Modelo de Persistencia Transaccional bajo notación Crow’s Foot.*

Figura 7. Diagrama de Máquina de Estados FSM (Dual)

La **Figura 7** ilustra el **diagrama de máquina de estados (State Machine / FSM)** dual del sistema: en la partición superior modela el ciclo de vida de los activos de red (`NetworkNode`), transitando desde el estado normal hasta el bloqueo perimetral, la infección por movimiento lateral o el aislamiento físico en cuarentena; en la partición inferior modela el autómata de toma de decisiones del adversario simulado (`Adversary AI FSM`), gobernado por la detección de aristas cerradas, el consumo de exploits `Bypass` y la contención total hacia los estados terminales.

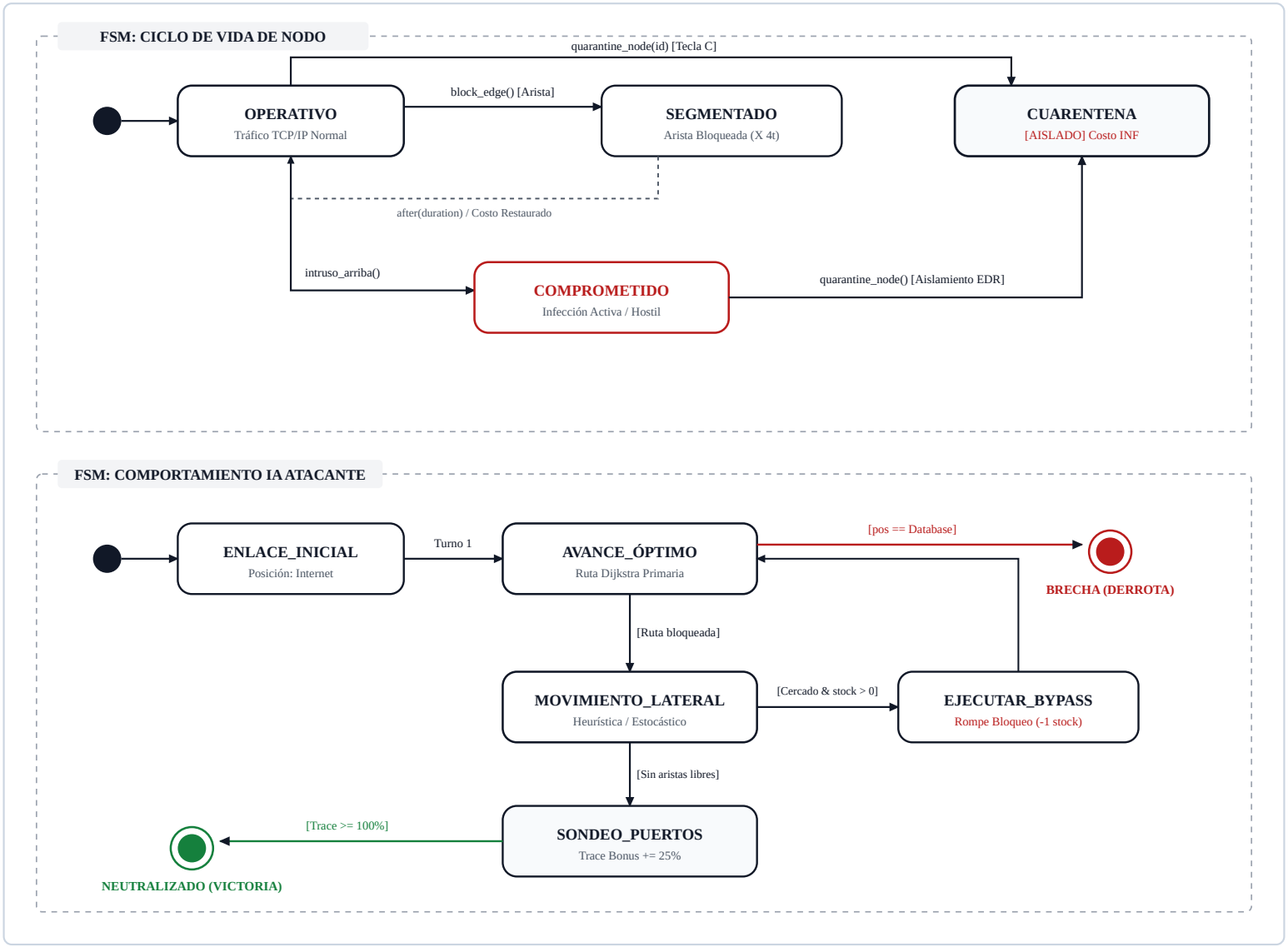


Figura 7. *Diagrama de estados (FSM) del ciclo de vida de los nodos y del autómata reactivo del adversario.*

Diccionario de Datos: Persistencia Transaccional (ANSI/IEEE Std 1059)

El presente diccionario de datos documenta formalmente la arquitectura de almacenamiento de **VERTEX** basada en el motor de serialización `ConfigFile` en la ruta de espacio de usuario `user://`. La persistencia implementa un desacoplamiento modular en tres ranuras (`slot_1`, `slot_2`, `slot_3`), gobernadas por el archivo de control maestro `slots.cfg` y respaldos automáticos en `backups/`.

Catálogo de Entidades y Archivos de Persistencia

Entidad / Tabla	Archivo Físico	Ámbito	Descripción Funcional
`SlotMeta`	`user://slots.cfg`	Global	Puntero maestro de ranura activa y banderas de migración heredada.
`SaveSlot`	`user://slots/slot_X/`	Ranura	Contenedor de ranura (1..3) con metadatos de integridad y caché de estrellas.
`Profile`	`.../profile.cfg`	Ranura	Identidad del analista (alias, icono y color de acento de interfaz).
`LevelProgress`	`.../progress.cfg`	Ranura	Registro de estrellas obtenidas y menor coste de camino por escenario.
`GlobalStats`	`.../stats.cfg`	Ranura	Métricas históricas agregadas (victorias, derrotas, racha y tiempo de juego).
`LevelStats`	`.../stats.cfg`	Ranura	Desglose discreto de victorias y derrotas por nivel táctico.
`BadgeRecord`	`BadgeRegistry`	Memoria	Catálogo de méritos, condecoraciones y avance hacia metas operacionales.
`SlotSnapshot`	`.../backups/`	Ranura / FS	Instantáneas atómicas rotativas (máx. 5) previas a borrados o modificaciones.

1. Entidad: `SlotMeta` (Metadatos Globales de Ranuras)

Archivo: `user://slots.cfg` | **Sección:** `[slots]`

Campo / Clave	Tipo	Rango	Nulidad	Clave	Por Defecto	Regla de Negocio / Descripción
`config_id`	String	32 chars	NOT NULL	**PK**	`meta_global`	Identificador maestro del archivo.
`active_slot`	int	1 .. 3	NOT NULL	Atrib	1	Índice de ranura activa del operador.
`migrated_legacy`	bool	0 / 1	NOT NULL	Atrib	false	Bandera de migración atómica a slot_1.

2. Entidad: `SaveSlot` (Contenedor de Ranura)

Ubicación: `user://slots/slot_X/` (donde $X \in \{1, 2, 3\}$)

Campo / Clave	Tipo	Rango	Nulidad	Clave	Por Defecto	Regla de Negocio / Descripción
`slot_id`	int	1 .. 3	NOT NULL	**PK**	N/A	Identificador ordinal de la ranura.
`is_empty`	bool	0 / 1	NOT NULL	Atrib	true	Indica ausencia de progreso y perfil.
`updated_at`	int	Epoch 64b	NOT NULL	Atrib	0	Marca de tiempo Unix de última modificación.
`cached_stars`	int	0 .. 45	NOT NULL	Atrib	0	Caché de estrellas para lectura rápida en menú.
`slot_path`	String	128 chars	NOT NULL	Atrib	`user://slots/slot_X`	Ruta normalizada en el sandbox de Godot.

Diccionario de Datos: Perfil, Progreso, Métricas y Respaldo

3. Entidad: `Profile` (Identidad del Analista)

Archivo: `user://slots/slot_X/profile.cfg` | **Sección:** `[profile]`

Campo / Clave	Tipo	Rango	Nulidad	Clave	Por Defecto	Regla de Negocio / Descripción
`profile_id`	String	36 chars	NOT NULL	**PK**	UUID	Identificador único de perfil en la ranura.
`slot_id`	int	1 .. 3	NOT NULL	**FK**	N/A	Referencia a SaveSlot.slot_id.
`player_name`	String	1..32 ch	NOT NULL	Atrib	`Operador`	Alias del analista (alfanumérico sanitizado).
`avatar_icon`	String	32 chars	NOT NULL	Atrib	`agent_blue`	Identificador del glifo de avatar táctico.
`theme_color`	String	7 chars	NOT NULL	Atrib	`#00E5FF`	Hexadecimal para acentos visuales y HUD.

4. Entidad: `LevelProgress` (Progresión y Puntuación)

Archivo: `user://slots/slot_X/progress.cfg` | **Sección:** `[estrellas]`

Campo / Clave	Tipo	Rango	Nulidad	Clave	Por Defecto	Regla de Negocio / Descripción
`level_key`	String	32 chars	NOT NULL	**PK (Comp)**	N/A	Clave del escenario (ej. "cyber_n1", "heist_n2").
`slot_id`	int	1 .. 3	NOT NULL	**FK (Comp)**	N/A	Vinculación a SaveSlot.slot_id.
`stars`	int	0 .. 3	NOT NULL	Atrib	0	Estrellas de maestría según optimización.
`mejor_coste`	float	Float 32	NOT NULL	Atrib	0.0	Menor peso acumulado de ruta (Dijkstra).
`completed`	bool	0 / 1	NOT NULL	Atrib	false	stars > 0; habilita acceso al siguiente nivel.

5. Entidades de Estadísticas: `GlobalStats` y `LevelStats`

Archivo: `user://slots/slot_X/stats.cfg` | **Secciones:** `[stats]` y `[levels]`

Campo / Clave	Tipo	Rango	Nulidad	Clave	Por Defecto	Regla de Negocio / Descripción
`total_wins`	int	≥ 0	NOT NULL	Atrib	0	Misiones culminadas con éxito (ambos bandos).
`total_losses`	int	≥ 0	NOT NULL	Atrib	0	Intentos fallidos (brechas o capturas).
`best_streak`	int	≥ 0	NOT NULL	Atrib	0	Máxima racha consecutiva de victorias.
`play_time`	int	≥ 0	NOT NULL	Atrib	0	Tiempo de enlace total en segundos.
`wins / losses`	int	≥ 0	NOT NULL	Atrib	0	Desglose discreto en sección [levels] por nivel.

6. Entidades de Soporte: `BadgeRecord` y `SlotSnapshot`

Campo / Clave	Tipo	Rango	Nulidad	Clave	Por Defecto	Regla de Negocio / Descripción
`badge_id`	String	32 chars	NOT NULL	**PK**	N/A	Identificador canónico de condecoración militar.
`unlocked`	bool	0 / 1	NOT NULL	Atrib	false	Estado de desbloqueo evaluado por BadgeRegistry.
`snapshot_name`	String	64 chars	NOT NULL	**PK**	N/A	Nombre de carpeta (snapshot_ts_tag).
`tag / timestamp`	Str / int	Varios	NOT NULL	Atrib	`auto`	Metadatos de rotación y captura atómica (máx. 5).